Gradient Descent and Its Variants

Machine Learning-2 — IIT Jodhpur, Lecture 4

Formatted by: Heisenber 😎

Index

- 1. The Story So Far — a hiker, a foggy valley, and one very important dial
- 2. Why Walking Downhill Actually Works — the Taylor series does the heavy lifting
- 3. The Algorithm, Written Down — four lines that run half of modern AI
- 4. The Learning Rate: Goldilocks in One Dimension — too small, too big, just right
- 5. A Two-Dimensional Walk to the Bottom — arithmetic you can check by hand
- 6. A Detour: Climbing Instead of Descending — logistic regression wants a maximum, not a minimum
- 7. The Bill Comes Due: Computational Cost — why "just sum over all the data" doesn't scale
- 8. Stochastic Gradient Descent — Betting on One Data Point — noisy, cheap, and it still works
- 9. The Jitters Near the Bottom — and the dynamic learning rate that calms them
- 10. The Ravine Problem — one learning rate, two very different slopes
- 11. Momentum — Don't Lose the Momentum — aggregate the past instead of forgetting it
- 12. AdaGrad — A Personal Learning Rate for Every Feature — frequent features get told to slow down
- 13. RMSProp — AdaGrad Without the Amnesia-in-Reverse — fixing an accumulator that never forgets
- 14. Adam — Everyone's Favourite Combo — momentum and RMSProp shake hands
- The Family Tree — one picture, seven algorithms
- Think-It-Through Questions — five questions worth losing sleep over

---

1. The Story So Far

Picture a hiker standing somewhere on a vast, hilly landscape, trying to reach the lowest point of a valley in thick fog. She cannot see the valley floor. All she can feel is the slope of the ground directly under her feet. Her only sensible strategy: feel which way is steepest, take a step downhill, and repeat.

That landscape is a function $f : \mathrm{R}^n \to \mathrm{R}$ — it eats a vector (her position, or in machine learning, a set of model parameters) and returns a single number (her altitude, or in machine learning, a loss). The "feel of the slope" is the **gradient**,

$$\nabla f = [\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \cdots \quad \frac{\partial f}{\partial x_n}]^\top$$

a vector stacking every partial derivative, one per coordinate. In a neural network these coordinates aren't abstract $x_1, x_2, \ldots$ — they are the network's actual learnable weights, and f is the loss computed from its predictions.

A closely related idea is the **directional derivative** — how fast f changes if the hiker steps in some specific direction $\mathbf{v}$ (not necessarily the steepest one):

$$D_{\mathbf{v}} f(\mathbf{x}) = \mathbf{v}^\top \nabla f(\mathbf{x}) = \cos\theta$$

where θ is the angle between $\mathbf{v}$ and $\nabla f(\mathbf{x})$. This is maximized — value 1 — exactly when $\theta = 0°$, i.e. when $\mathbf{v} = \nabla f(\mathbf{x})$:

$$\max_{\mathbf{v}} \ \mathbf{v}^\top \nabla f(\mathbf{x}) = 1 \quad \text{when } \theta = 0^\circ \text{ or } \mathbf{v} = \nabla f(\mathbf{x})$$

In plain words: **the gradient points in the direction of steepest ascent.** So if the hiker wants to go *up* fastest, she follows ∇f; if she wants to go *down* — which is what minimizing a loss means — she follows $-\nabla f$. Every algorithm in this lecture is a variation on that one idea.

> **Note for Review:** Students often think the gradient "points toward the minimum." It doesn't — it only points in the direction of steepest local *increase*. It has no idea where the minimum is; it only knows which way is uphill *right here*. That's precisely why gradient descent is an iterative, step-by-step algorithm rather than a one-shot formula.

2. Why Walking Downhill Actually Works

This isn't just a good metaphor — it's a theorem, and a short one. Let $f : \mathrm{R}^n \to \mathrm{R}$ be a scalar function. The Taylor series expansion of f around a point $\mathbf{x}$, in a small neighbourhood $\boldsymbol{\epsilon}$, is

$$f(\mathbf{x} + \boldsymbol{\epsilon}) = f(\mathbf{x}) + \frac{\boldsymbol{\epsilon}^\top \nabla f}{1!} + \frac{1}{2!}\boldsymbol{\epsilon}^\top \nabla^2 f\, \boldsymbol{\epsilon} + \cdots$$

Now comes the one clever substitution that the entire lecture hinges on. Choose the step to be

$$\boldsymbol{\epsilon} = -\eta\, \nabla f, \qquad \eta > 0$$

Here η (the learning rate, usually written α) is deliberately kept positive — we are choosing to step in the *opposite* direction of the gradient. Substitute this into the Taylor expansion and drop the second-order-and-beyond terms (they vanish faster than the first order term as $\boldsymbol{\epsilon} \to 0$):

$$f(\mathbf{x} - \eta \nabla f) = f(\mathbf{x}) - \eta\, \nabla f^\top \nabla f = f(\mathbf{x}) - \eta\, \|\nabla f\|_2^2$$

Look closely at the term being subtracted: $\eta > 0$ and $\|\nabla f\|_2^2 \geq 0$, so their product is never negative. That single observation delivers the whole result:

$$f(\mathbf{x}) > f(\mathbf{x} - \eta \nabla f)$$

Translating back to the hiker: if her current position is $\mathbf{x}$, and her next position is $\mathbf{x} - \eta \nabla f(\mathbf{x})$, then it is *guaranteed* that the new position has a strictly lower altitude (as long as the gradient isn't exactly zero). This is the whole justification for the update rule

$$\mathbf{x} \;\longrightarrow\; \mathbf{x} - \eta\, \nabla f(\mathbf{x})$$

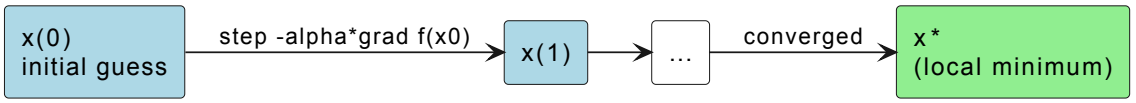
Repeat this enough times, and $f(\mathbf{x})$ keeps shrinking step after step, walking down toward a local minimum.

3. The Algorithm, Written Down

Algorithm — Gradient Descent

1. Initialize $\mathbf{x}^{(0)}$ and a step size α.
2. $t \leftarrow 0$
3. **while** not converged **do**
4. $\quad \mathbf{x}^{(t+1)} \leftarrow \mathbf{x}^{(t)} - \alpha\, \nabla f(\mathbf{x}^t)$
5. $\quad t \leftarrow t + 1$
6. **end while**

That's it — four working lines. Everything from here on is about making step 4 smarter: cheaper to compute, more stable, and better-behaved when the landscape has a nasty shape. Note the sign convention: it's a **minus** sign for minimization (descending), and it flips to a **plus** for maximization (ascending) — we'll need that flip in Section 6.

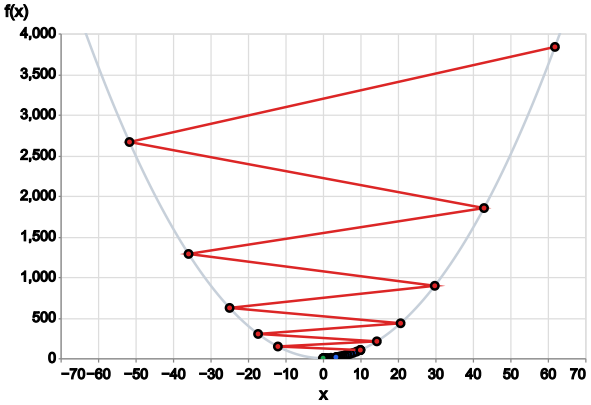


4. The Learning Rate: Goldilocks in One Dimension

Let's put a number on this before adding any more machinery. Take the simplest possible bowl-shaped function, $f(x) = x^2$, starting from $x_0 = 10$, and watch what three different values of α do to the same algorithm.

Because $\nabla f(x) = 2x$, the update rule is just $x^{(t+1)} = x^{(t)} - 2\alpha\, x^{(t)} = (1 - 2\alpha)\, x^{(t)}$ — a simple geometric shrinkage (or growth) with ratio $(1 - 2\alpha)$.

- $\alpha = 0.2$**:** ratio is 0.6. The sequence shrinks steadily: $10, 6, 3.6, 2.16, \ldots$ — it reaches essentially zero within about 10 steps.
- $\alpha = 0.05$**:** ratio is 0.9. Still shrinking, but sluggishly: $10, 9, 8.1, 7.29, \ldots$ — after the same 10 steps it has barely covered a third of the distance.
- $\alpha = 1.10$**:** ratio is -1.2. Its *magnitude* exceeds 1, so instead of shrinking, the sequence **grows** and flips sign every step: $10, -12, 14.4, -17.28, \ldots$ — by the 10th step $f(x)$ has ballooned from 100 to over 3800.



This little algebraic fact — the update is a repeated multiplication by $(1 - 2\alpha)$ — is the entire reason learning rate tuning matters so much in practice: whether that multiplier's magnitude is below 1 (converges), close to 1 (crawls), or above 1 (diverges) decides the entire fate of training. The rule of thumb that follows is exactly what you'd guess: **when unsure, start small.** A too-small α only costs you time; a too-large α can throw the whole optimization into an undefined, exploding mess.

---

5. A Two-Dimensional Walk to the Bottom

Everything above generalizes cleanly beyond one variable — the same Taylor-series argument from Section 2 didn't actually care what n was. Let's take a genuine 2D function and walk through the arithmetic by hand, the way you'd want to be able to redo it in an exam.

$$f(x_1, x_2) = x_1^2 + 2x_2^2, \qquad \nabla f = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

Setting $\nabla f = \mathbf{0}$ gives the unique global minimum at $x^* = (0, 0)$. Start at $\mathbf{x}^{(0)} = (-5, -2)$ with $\alpha = 0.1$.

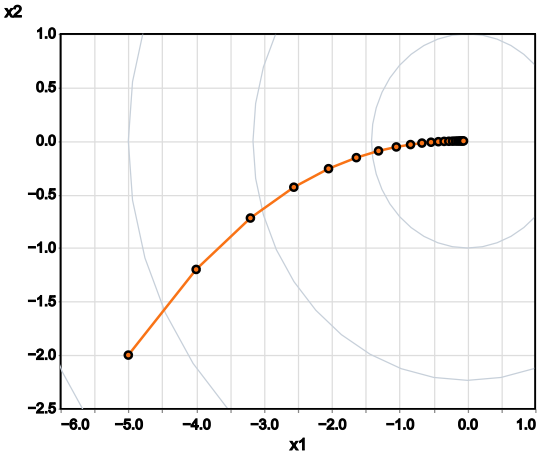
Iteration 1:

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} - 0.1\, \nabla f(\mathbf{x}^{(0)}) = \begin{bmatrix} -5 \\ -2 \end{bmatrix} - 0.1 \begin{bmatrix} -10 \\ -8 \end{bmatrix} = \begin{bmatrix} -5 \\ -2 \end{bmatrix} + \begin{bmatrix} 1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} -4 \\ -1.2 \end{bmatrix}$$

Iteration 2: now using the gradient *at the new point* $(-4, -1.2)$:

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} - 0.1\, \nabla f(\mathbf{x}^{(1)}) = \begin{bmatrix} -4 \\ -1.2 \end{bmatrix} - 0.1 \begin{bmatrix} -8 \\ -4.8 \end{bmatrix} = \begin{bmatrix} -4 \\ -1.2 \end{bmatrix} + \begin{bmatrix} 0.8 \\ 0.48 \end{bmatrix} = \begin{bmatrix} -3.2 \\ -0.72 \end{bmatrix}$$

Each point is closer to the origin than the last. Plotting several more iterations traces out a smooth curved path into the bowl:



Notice something worth remembering here: far from the minimum the gradient is large, so the steps are big; as the point nears $(0, 0)$ the gradient shrinks, and so do the steps — the path visibly slows down as it approaches the centre. That single observation — **step size is proportional to local gradient magnitude, not to distance-to-go** — is the seed of almost every problem discussed later in this lecture.

6. A Detour: Climbing Instead of Descending

Not every machine learning problem wants a minimum. Logistic regression, trained by maximum likelihood, wants to find parameters that make the observed labels *most probable* — that's a maximization problem, and the fix is exactly the sign flip promised in Section 3.

Given a training set $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$ and a sigmoid predictor $h_\theta(\mathbf{x}_i) = \dfrac{1}{1 + e^{-\theta^\top \mathbf{x}_i}}$, treated as a probability, the algorithm is:

Algorithm — Gradient Ascent for Logistic Regression

1. Initialize $\theta^{(0)}$ and learning rate α.
2. $t \leftarrow 0$
3. **while** not converged **do**
4. **for** $j \in \{0, 1, \dots, d\}$ **do**
5. $\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} + \alpha \sum_{i=1}^{m} (y_i - h_\theta(\mathbf{x}_i))\, x_{ij}$
6. **end for**
7. $t \leftarrow t + 1$
8. **end while**
9. **Output:** optimal classifier h_{θ^*}

The plus sign in step 5 is the only structural difference from Section 3's algorithm — everything else (initialize, iterate, update, converge) is the identical skeleton. This is worth internalizing: gradient descent and gradient ascent are the same walk, just facing opposite directions.

7. The Bill Comes Due: Computational Cost

Look again at step 5 above: that $\sum_{i=1}^{m}$ runs over **every single training example**, for **every single parameter**, at **every single iteration**. For a small toy dataset this is nothing. But real datasets have lakhs (or millions) of rows, and real models — deep networks especially — have equally huge numbers of parameters. Multiply "many data points" by "many parameters" by "many iterations," and full-batch gradient descent becomes computationally infeasible.

The general form of the loss function that causes this is:

$$f(x) = \frac{1}{n} \sum_{i=1}^{n} f_i(x)$$

which is exactly what shows up in something as ordinary as linear regression, where the loss is a mean squared error over the whole dataset:

$$\ell(\theta) = \frac{1}{n} \sum_{i=1}^{n} e_i^2 = \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{x}_i^\top \theta - y_i\|_2^2 = \frac{1}{n} \sum_{i=1}^{n} f_i(\mathbf{x}_i, \theta)$$

Differentiating a sum is the same as summing the derivatives, so the gradient inherits the exact same expensive summation:

$$\nabla_\theta\, \ell(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla f_i$$

and correspondingly the full gradient descent update becomes

$$x^{(t+1)} = x^{(t)} - \alpha \frac{1}{n} \sum_{i=1}^{n} \nabla_x f_i(x)$$

Nearly every loss function in machine learning is built this way — a mean over training examples — so this summation bottleneck isn't a special case, it's the default case.

8. Stochastic Gradient Descent — Betting on One Data Point

The fix is almost cheeky in its simplicity: **stop summing over everyone, and just use one randomly chosen data point to *estimate* the gradient.**

$$x^{(t+1)} = x^{(t)} - \alpha \nabla_x f_i(x)$$

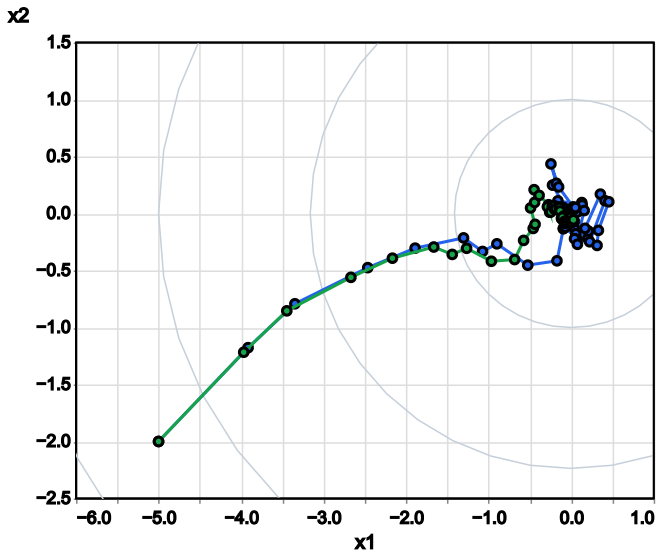
Compare this to the full-batch update above — the $\frac{1}{n}\sum_{i=1}^{n}$ has vanished entirely, replaced by a single term $\nabla_x f_i(x)$ for one randomly picked i. This single data point's gradient is a noisy estimate of the true, full-dataset gradient — it will not point in exactly the right direction every step — but on average, over many steps, it is unbiased enough to still walk the solution toward the minimum. This is what the word **stochastic** in "stochastic gradient descent" refers to: randomness in *which* data is used to estimate each step, not randomness in the algorithm's logic.

In practice, nobody actually uses a single data point — that under-uses the parallel power of a GPU or CPU. Instead, a small, uniformly-random **mini-batch** of the data is used per step, and the summation runs only over that batch. This is **mini-batch stochastic gradient descent**, and it is what almost every deep learning framework means by "SGD" today.

> **Note for Review:** "Stochastic" vs. "mini-batch" is often asked as a trick distinction — batch size 1 is stochastic GD, batch size m (the whole dataset) is full-batch GD, and anything strictly in between is mini-batch GD. They all share the exact same update equation; only the size of the averaging set changes.

9. The Jitters Near the Bottom

Running SGD on the same $f(x_1, x_2) = x_1^2 + 2x_2^2$ bowl, starting again from $(-5, -2)$, with the *same* learning rate $\alpha = 0.1$ used before, produces a path that looks identical to full-batch gradient descent while it's far from the minimum — but something different happens as it arrives:



Far from the minimum, the single-point (or mini-batch) gradient estimate is close enough to the true gradient that the path tracks the smooth full-batch curve almost exactly (blue). But close to the minimum, the *true* gradient is tiny — while the noise in a one-point estimate stays roughly the same size regardless of location. Signal shrinks, noise doesn't: the ratio flips, and the path starts to visibly jitter around the minimum instead of settling into it, as the constant-rate blue path shows.

There are two fixes. One is simply a bigger batch size, which averages out more of the noise per step. The other is a clever trick called the **dynamic (or decaying) learning rate**: instead of keeping α fixed for the whole run, shrink it over time,

$$\alpha(t) = \alpha_0 \, e^{-\lambda t}$$

where t is the iteration number. Early on, when big steps are safe and useful, $\alpha(t) \approx \alpha_0$ is large. Later, as the point nears the optimum and jitter becomes the dominant risk, $\alpha(t)$ has decayed toward zero, damping the noise-driven fluctuations. The green path above uses exactly this decay schedule, and visibly settles down rather than dancing around the minimum.

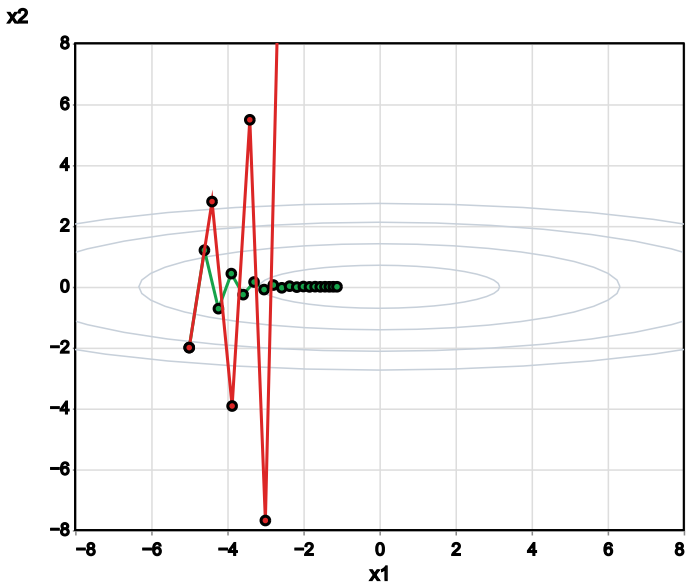
10. The Ravine Problem

So far every direction in our example bowl was "equally curved" — x_1 and x_2 misbehaved by the same amount. Real loss surfaces are rarely so polite. Some features cause the loss to change very sharply; others barely move it at all. Model that asymmetry with a lopsided bowl:

$$f(x_1, x_2) = 0.1x_1^2 + 2x_2^2, \qquad \nabla f = \begin{bmatrix} 0.2x_1 \\ 4x_2 \end{bmatrix}$$

Evaluate the gradient at, say, $(10, 10)$ with a learning rate α: the step in the x_1 direction is $\alpha \cdot 0.2 \cdot 10 = 2\alpha$, while the step in the x_2 direction is $\alpha \cdot 4 \cdot 10 = 40\alpha$ — a **twenty-fold** difference, using the very same α. Along x_1 you'll crawl forward; along x_2 you'll leap. There is no single value of α that is simultaneously "safe" for the steep direction and "fast enough" for the shallow one.

Watch what happens when you try anyway, using the *same* α for both coordinates:



At $\alpha = 0.4$ (green), the path oscillates back and forth across the steep x_2 direction while slowly crawling along the shallow x_1 direction — inefficient, but it converges. Push α up to just 0.6 (red) — a change of only 0.2 — and the *same* oscillation along x_2 now overshoots further each time instead of dying down; within about 15 iterations x_2 has exploded from -2 toward the thousands, and the algorithm has diverged. A single scalar learning rate simply cannot serve two very differently-scaled directions at once — and this exact situation is common in real models, where different features (or different layers of a network) naturally have very different gradient scales.

11. Momentum — Don't Lose the Momentum

The fix has a memorable name and a memorable idea: **don't lose the momentum.** Instead of using only the gradient at the *current* point to decide the next step, aggregate the gradients from several previous steps as well:

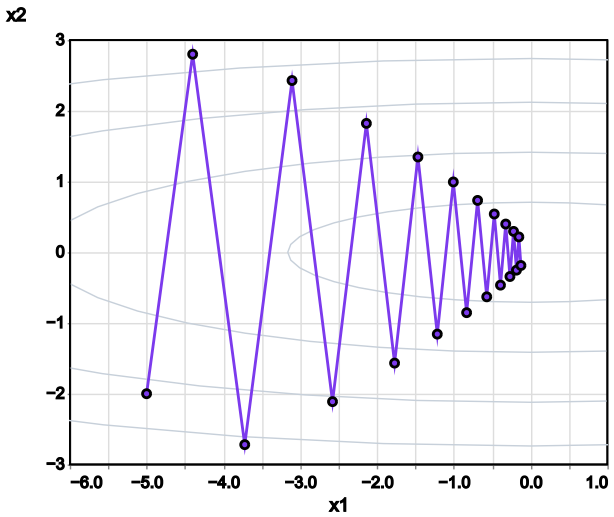
$$\mathbf{v}_t = \beta\, \mathbf{v}_{t-1} + \mathbf{g}_{t,t-1}, \qquad \text{where } \mathbf{g}_{t,t-1} = \nabla_{\mathbf{x}} \frac{1}{|\mathrm{B}_t|} \sum_{i \in \mathrm{B}_t} f_i(\mathbf{x}_{t-1})$$

$$\mathbf{x}_t = \mathbf{x}_{t-1} - \alpha_t\, \mathbf{v}_t$$

Here $\mathbf{g}_{t,t-1}$ is simply the (mini-batch) gradient at the current step, and $\mathbf{v}_t$ is a running, exponentially-weighted aggregate of gradients — the "velocity." Why does aggregating gradients fix *both* of the Section 10 problems at once?

- **Along the shallow direction** (x_1 in our ravine), the gradient always points the same way, iteration after iteration. Summing several same-signed small numbers gives a noticeably larger number — so the aggregated step is bigger than any single raw gradient, and progress speeds up.
- **Along the steep direction** (x_2), the gradient keeps flipping sign as the path overshoots back and forth. Summing a positive and a negative largely **cancels them out**, shrinking the aggregated step and damping the oscillation.

Running momentum with $\alpha = 0.6$ and $\beta = 0.25$ on the same ravine that made plain SGD diverge:



The zig-zag is still visible early on — momentum doesn't eliminate oscillation instantly — but each swing is noticeably smaller than the last, and the path settles in near the origin instead of blowing up. Compare this to the $\alpha = 0.6$ divergence in Section 10: same learning rate along the dangerous direction, but a fundamentally more stable outcome, purely because the update remembers its recent past instead of reacting only to the instantaneous gradient.

---

12. AdaGrad — A Personal Learning Rate for Every Feature

Momentum solves the ravine problem by looking backward at *recent* gradients. **AdaGrad** (Adaptive Gradient) takes a different angle: instead of one shared α for every coordinate, give **each parameter its own effective learning rate**, based on how large its gradients have historically been.

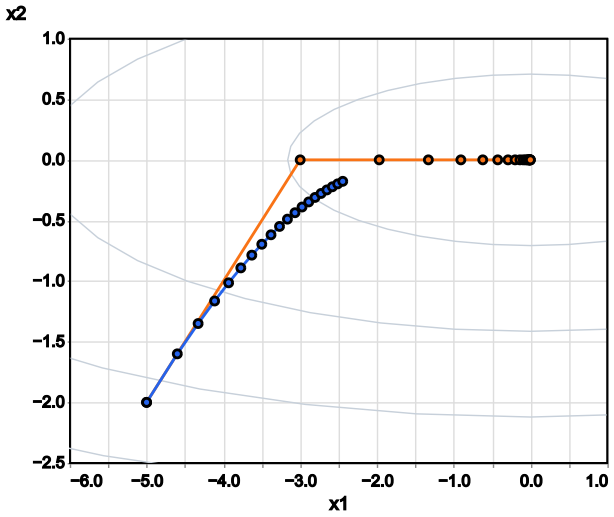
$$\mathbf{g}_t = \nabla_{\mathbf{x}}\, \ell(y_t, f(\mathbf{x}_t, \boldsymbol{\omega}))$$

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t^2$$

$$\boldsymbol{\omega}_t = \boldsymbol{\omega}_{t-1} - \frac{\alpha}{\sqrt{\mathbf{s}_t + \epsilon}}\, \mathbf{g}_t$$

All operations here are element-wise: $\mathbf{s}_t$ accumulates the *squared* gradient magnitude seen so far, separately for every coordinate (ϵ is just a small safety constant so the division never blows up if a coordinate's gradient has been exactly zero throughout). A coordinate whose gradients have historically been large builds up a large s_t, which shrinks $\alpha/\sqrt{s_t + \epsilon}$ — effectively slowing that coordinate down. A coordinate with historically tiny gradients keeps its effective rate closer to the full α.

This maps directly onto how real features behave: features that appear very frequently in the training data tend to accumulate large gradients and get automatically throttled, while rare, infrequent features keep receiving comparatively large updates so they aren't drowned out.



With $\alpha = 0.4$ (blue), convergence along the shallow x_1 axis is still fairly slow — the accumulator hasn't grown large enough yet to make a big difference. Push α up to 2.0 (orange) and the path converges dramatically faster, without the instability that plain SGD showed at high learning rates in Section 10 — because AdaGrad is automatically damping each coordinate according to its own accumulated gradient history.

13. RMSProp — AdaGrad Without the Amnesia-in-Reverse

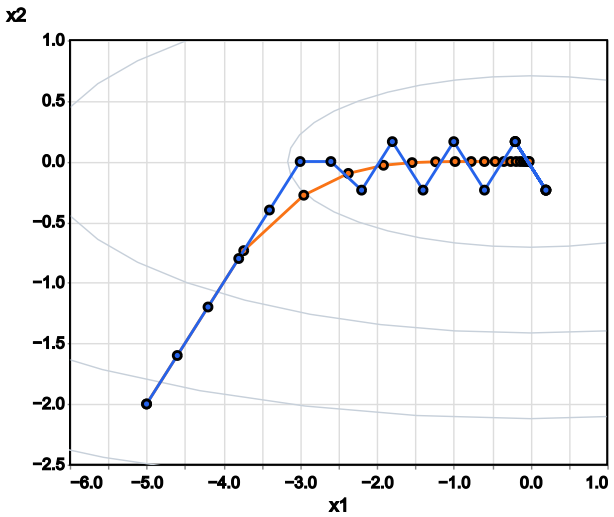
AdaGrad has a subtle flaw hiding in plain sight: s_t is a **running sum that never shrinks** — every squared gradient ever seen stays in the accumulator forever, so s_t grows without bound over a long training run. Eventually $\alpha/\sqrt{s_t + \epsilon}$ decays so close to zero that learning effectively stalls, even if the model is still far from optimal.

RMSProp (root mean square propagation) fixes this with one small but powerful change: replace the plain running sum with an **exponentially decayed moving average**, so old gradients are gradually forgotten instead of accumulating forever.

$$\mathbf{s}_t = \gamma\, \mathbf{s}_{t-1} + (1-\gamma)\, \mathbf{g}_t^2$$

$$\boldsymbol{\omega}_t = \boldsymbol{\omega}_{t-1} - \frac{\alpha}{\sqrt{\mathbf{s}_t + \epsilon}}\, \mathbf{g}_t$$

The update rule for ω_t is *identical* to AdaGrad's — the only change is how s_t is computed. With γ close to 1 (a common choice is $\gamma = 0.9$), recent gradients dominate the average while older ones fade away, so s_t stays bounded instead of growing forever.



With no decay ($\gamma = 0$, blue — behaving like a very short-memory AdaGrad) the path keeps a persistent small oscillation near the minimum, since each fresh gradient dominates s_t completely. With $\gamma = 0.9$ (orange), the accumulator smooths over several recent steps, damping that oscillation into a tighter, calmer path toward the origin.

14. Adam — Everyone's Favourite Combo

If Momentum's idea is "remember recent gradients to smooth direction," and RMSProp's idea is "remember recent squared gradients to adapt the rate," the natural next question is: **why not do both?** That combination is **Adam** (Adaptive Moment Estimation):

$$\mathbf{g}_t = \nabla_{\mathbf{x}}\, \ell(y_t, f(\mathbf{x}_t, \boldsymbol{\omega}))$$

$$\mathbf{v}_t = \beta_1\, \mathbf{v}_{t-1} + (1-\beta_1)\, \mathbf{g}_t \qquad \text{(momentum — 1st moment)}$$

$$\mathbf{s}_t = \beta_2\, \mathbf{s}_{t-1} + (1-\beta_2)\, \mathbf{g}_t^2 \qquad \text{(RMSProp — 2nd moment)}$$

Both $\mathbf{v}_t$ and $\mathbf{s}_t$ start at zero, which biases them toward zero especially in the first few iterations. Adam corrects for this with a **bias-correction** step:

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1-\beta_2^t}$$

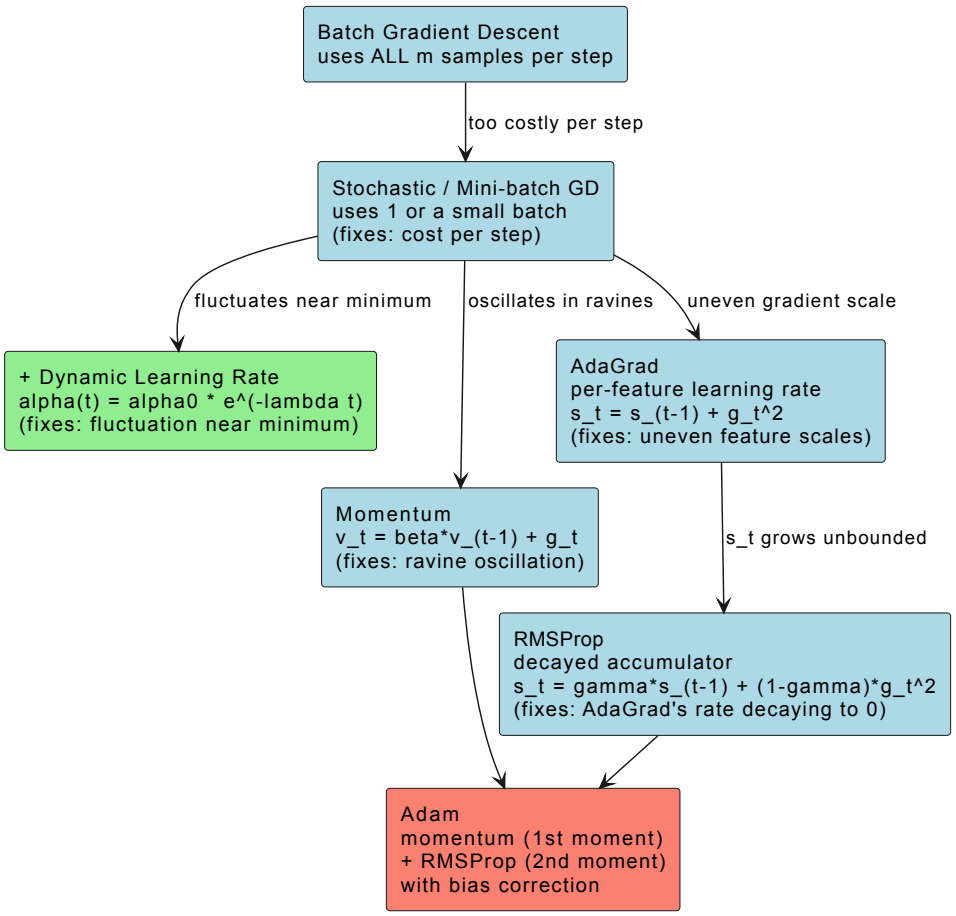
and finally updates the parameters using the corrected momentum in the numerator and the corrected squared-gradient average in the denominator:

$$\boldsymbol{\omega}_t = \boldsymbol{\omega}_{t-1} - \frac{\alpha\, \hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t + \epsilon}}$$

One subtlety worth noticing: $\mathbf{v}_t$ aggregates the *signed* gradient (so opposite-signed gradients along an oscillating direction cancel, exactly as in plain Momentum), while $\mathbf{s}_t$ aggregates the *squared* gradient (so it only ever measures magnitude, never direction, exactly as in RMSProp). Adam takes each algorithm's strength and refuses to give up the other's.

The Family Tree

Zooming out, every variant in this lecture exists to patch one specific weakness of the previous one:



Think-It-Through Questions

1. In Section 4 the update on $f(x) = x^2$ reduced to $x^{(t+1)} = (1 - 2\alpha)x^{(t)}$. For a general quadratic $f(x) = cx^2$ with $c > 0$, work out the exact condition on α (in terms of c) that separates convergence from divergence. Now connect this back to Section 10: why does an *elongated* bowl (very different curvature along different axes) make it fundamentally impossible to choose one α that is safe for every direction at once?

2. Momentum (Section 11) and RMSProp (Section 13) both maintain a running aggregate of gradient information, yet one aggregates $\mathbf{g}_t$ while the other aggregates $\mathbf{g}_t^2$. Construct a small thought experiment — a gradient sequence over a few steps — where this difference actually matters: pick a scenario where Momentum would speed things up but RMSProp's accumulator would barely change, or vice versa.

3. AdaGrad's accumulator s_t only ever grows. Suppose training runs for a very large number of iterations on a feature that had a handful of huge gradients early on (say, during the first few steps) and then contributes only small gradients for the rest of training. What happens to that feature's effective learning rate for the remainder of training, and is that behaviour actually *desirable* or a hidden flaw? How does RMSProp's design change your answer?

4. The dynamic learning rate in Section 9 decays *purely as a function of iteration number* t, regardless of how close the model actually is to the optimum. Adam's bias correction, by contrast, is also a function of t (through β_1^t and β_2^t), but for a completely different reason. Explain precisely what each of these two "functions of t" is compensating for, and why neither one is aware of the model's actual distance from the minimum.

5. Suppose you are training a model where one group of features is extremely common (appears in nearly every training example) and another group is extremely rare (appears in less than 1% of examples), and suppose the loss surface also happens to have a momentum-favouring ravine shape between two of the common features. Would you reach for AdaGrad/RMSProp, Momentum, or Adam — and justify your choice using the *specific mechanism* (not just the name) of whichever algorithm(s) you pick. Is there a scenario in this setup where your chosen algorithm could still fail?